UNIVERSIDAD MARIANO GALVEZ DE GUATEMALA

Centro Universitario Portales Domingo, Guatemala · Ingenieria en Sistemas

PROGRAMACION 2 · DOCUMENTACION TECNICA

Sistema de Prestamos de Biblioteca

Informe academico y anexo de codigo fuente en C# con SQLite

Circuito completo de base de datos

Proyecto	Sistema de Prestamos de Biblioteca
Lenguaje / Plataforma	C# / .NET 6 (aplicacion de consola)
Motor de base de datos	SQLite mediante Microsoft.Data.Sqlite 6.0.36
Alcance	Tablas, CRUD, prestamos y devoluciones
Asignatura	Programacion 2
Fecha	09 de octubre de 2026

Documento generado automaticamente a partir del codigo fuente del proyecto. Incluye la totalidad de los archivos.

1. Resumen Ejecutivo

El presente informe documenta el proyecto **Sistema de Prestamos de Biblioteca**, desarrollado en lenguaje C# sobre la plataforma .NET 6 y con persistencia en una base de datos SQLite. Su proposito didactico es recorrer de forma completa el circuito de una base de datos: identificar entidades y relaciones, modelar las tablas, conectarse al motor desde el codigo, implementar las operaciones CRUD (crear, consultar, actualizar y eliminar) y aplicar reglas de negocio con manejo de errores controlado.

El sistema administra tres entidades relacionadas: **Lectores**, **Libros** y **Prestamos**. Un prestamo asocia un lector con un libro e incluye las fechas de inicio, limite y devolucion. El sistema controla la disponibilidad de ejemplares, impide prestamos sin existencias y calcula el estado de cada prestamo (Prestado, Vencido o Devuelto).

11	3	4	620
ARCHIVOS	TABLAS	CAPAS	LINEAS DE CODIGO

2. Objetivos

- Modelar un caso de uso real mediante tablas relacionadas con claves foraneas.
- Implementar tecnicas de Programacion Orientada a Objetos: abstraccion, herencia, polimorfismo y encapsulamiento.
- Aplicar el patron de separacion por capas: dominio, persistencia y aplicacion.
- Ejecutar operaciones CRUD seguras usando consultas parametrizadas y transacciones.
- Controlar errores de entrada y de negocio mediante `try-catch` sin cerrar la aplicacion.
- Distribuir el proyecto de forma portable en SQLite, sin requerir un servidor de base de datos.

3. Arquitectura y Estructura

El proyecto se organiza en cuatro capas que separan las responsabilidades y facilitan el mantenimiento:

Dominio

Clases de negocio: `Persona`, `Lector`, `Libro` y `Prestamo`. Contienen las reglas y validaciones.

Persistencia

`InicializadorBaseDatos` crea la base y las tablas; `RepositorioBiblioteca` ejecuta el CRUD sobre SQLite.

Aplicacion

`MenuBiblioteca` presenta las opciones, captura datos y muestra resultados en consola.

Configuracion

`Program.cs` y el archivo `.csproj` arrancan la aplicacion y declaran la dependencia de SQLite.

Inventario de archivos del codigo

Archivo	Capa / Rol

`PrestamosBiblioteca.csproj`	Configuracion del proyecto
`Program.cs`	Punto de entrada
`Dominio/Persona.cs`	Dominio (POO)
`Dominio/Lector.cs`	Dominio (POO)
`Dominio/Libro.cs`	Dominio (POO)
`Dominio/Prestamo.cs`	Dominio (POO)
`Persistencia/InicializadorBaseDatos.cs`	Persistencia (SQLite)
`Persistencia/RepositorioBiblioteca.cs`	Persistencia (SQLite)
`Aplicacion/MenuBiblioteca.cs`	Aplicacion (consola)
`BaseDatos/crear_tablas.sql`	Base de datos
`README.md`	Documentacion

4. Diseno de la Base de Datos

La base de datos SQLite `biblioteca.db` se crea automaticamente al iniciar la aplicacion y se ubica en `%LOCALAPPDATA%\UMG\PrestamosBiblioteca\`. El esquema se compone de tres tablas.

Tabla	Campos principales	Clave / Restricciones
Lectores	Id, Codigo, Nombre, Correo, Telefono	Clave primaria Id; Codigo y Correo unicos.
Libros	Id, Codigo, Titulo, Autor, Ejemplares	Clave primaria Id; Codigo unico; Ejemplares >= 0.
Prestamos	Id, LectorId, LibroId, FechaPrestamo, FechaLimite, FechaDevolucion	Claves foraneas a Lectores y Libros; validacion de fechas; borrado en cascada.

Relaciones

- **Lector 1 --- N Prestamo:** un lector puede tener varios prestamos.
- **Libro 1 --- N Prestamo:** un libro puede registrarse en varios prestamos a lo largo del tiempo.
- Los ejemplares **disponibles** de un libro equivalen a sus `Ejemplares` totales menos los prestamos activos (sin fecha de devolucion).

> La opcion **ON DELETE CASCADE** elimina automaticamente los prestamos asociados cuando se borra un lector o un libro, preservando la integridad referencial. Las consultas usan parametros (`$nombre`) para evitar la inyeccion SQL.

5. Reglas de Negocio y Flujo

- No se registra un prestamo si el libro no tiene ejemplares disponibles.
- La fecha limite no puede ser anterior a la fecha del prestamo.

- La devolucion no puede ser anterior a la fecha del prestamo.
- Un ejemplar vuelve a estar disponible al registrar su devolucion.
- Los ejemplares de un libro no pueden reducirse por debajo de los prestamos activos.
- El estado del prestamo se determina como `Devuelto`, `Vencido` o `Prestado`.

Flujo principal

Registrar lector y libro → crear prestamo (valida disponibilidad) → consultar prestamos → registrar devolucion (libera el ejemplar) → consultar disponibilidad. Todo error se captura con `try-catch` y se informa al usuario.

6. Verificacion Realizada

- Compilacion con `dotnet build`: 0 advertencias y 0 errores.
- Restauracion automatica de `Microsoft.Data.Sqlite` desde NuGet al ejecutar `dotnet restore`.
- Pruebas de CRUD de lectores, libros y prestamos.
- Rechazo controlado del prestamo cuando no hay ejemplares disponibles.
- Registro de devolucion y liberacion del ejemplar (estado `Devuelto`).
- Actualizaciones y eliminaciones con integridad referencial.
- Demostracion de herencia y polimorfismo mediante `Persona = new Lector(...)`.

7. Indice de Codigo Fuente

Cada archivo se presenta a continuacion numerado y con la totalidad de su contenido. Se incluyen los archivos de configuracion, las clases de dominio, la capa de persistencia, la capa de aplicacion, el script SQL y la documentacion.

1. `PrestamosBiblioteca.csproj` [Configuracion del proyecto]
2. `Program.cs` [Punto de entrada]
3. `Dominio/Persona.cs` [Dominio (POO)]
4. `Dominio/Lector.cs` [Dominio (POO)]
5. `Dominio/Libro.cs` [Dominio (POO)]
6. `Dominio/Prestamo.cs` [Dominio (POO)]
7. `Persistencia/InicializadorBaseDatos.cs` [Persistencia (SQLite)]
8. `Persistencia/RepositorioBiblioteca.cs` [Persistencia (SQLite)]
9. `Aplicacion/MenuBiblioteca.cs` [Aplicacion (consola)]
10. `BaseDatos/crear_tablas.sql` [Base de datos]
11. `README.md` [Documentacion]

8. Codigo Fuente Completo

Las secciones siguientes contienen el codigo fuente integro del proyecto, organizado en el mismo orden del indice.

ARCHIVO 1 DE 11

PrestamosBiblioteca.csproj

Configuracion del proyecto

Proyecto .NET 6 de consola. Declara el paquete NuGet Microsoft.Data.Sqlite 6.0.36 (que aporta el proveedor SQLite y sus dependencias) y embebe el script de tablas como recurso.

```
<Project Sdk="Microsoft.NET.Sdk">
  <PropertyGroup>
    <OutputType>Exe</OutputType>
    <TargetFramework>net6.0</TargetFramework>
    <ImplicitUsings>enable</ImplicitUsings>
    <Nullable>disable</Nullable>
  </PropertyGroup>

  <ItemGroup>
    <PackageReference Include="Microsoft.Data.Sqlite" Version="6.0.36" />
    <EmbeddedResource Include="BaseDatos\crear_tablas.sql" />
  </ItemGroup>
</Project>
```

ARCHIVO 2 DE 11

Program.cs

Punto de entrada

Inicia la aplicacion: crea la base de datos, construye el repositorio y arranca el menu. Envuelve el arranque en try-catch.

```
using PrestamosBiblioteca.Aplicacion;
using PrestamosBiblioteca.Persistencia;

namespace PrestamosBiblioteca;

internal static class Program
{
    private static void Main()
    {
        try
        {
            var inicializador = new InicializadorBaseDatos();
            inicializador.Inicializar();
            var repositorio = new RepositorioBiblioteca(inicializador.CadenaConexion);
            new MenuBiblioteca(repositorio).Ejecutar();
        }
        catch (Exception error)
        {
            Console.ForegroundColor = ConsoleColor.Red;
            Console.WriteLine($"No fue posible iniciar el sistema: {error.Message}");
            Console.ResetColor();
        }
    }
}
```

ARCHIVO 3 DE 11

Dominio/Persona.cs

Dominio (POO)

Clase abstracta base con codigo, nombre y correo. Define el metodo abstracto ObtenerTipo y la validacion de texto comun.

```
namespace PrestamosBiblioteca.Dominio;

public abstract class Persona
{
    protected Persona(string codigo, string nombre, string correo)
    {
        Codigo = ValidarTexto(codigo, "El codigo del lector es obligatorio.");
        Nombre = ValidarTexto(nombre, "El nombre es obligatorio.");
        Correo = ValidarTexto(correo, "El correo es obligatorio.");
    }

    public string Codigo { get; }
    public string Nombre { get; }
    public string Correo { get; }

    public abstract string ObtenerTipo();
    public virtual string ObtenerInformacion() => $"{ObtenerTipo()}: {Codigo} - {Nombre} | {Correo}";

    protected static string ValidarTexto(string valor, string mensaje) =>
        string.IsNullOrWhiteSpace(valor) ? throw new ArgumentException(mensaje) : valor.Trim();
}
```

ARCHIVO 4 DE 11

Dominio/Lector.cs

Dominio (POO)

Hereda de Persona. Agrega el telefono y sobrescribe ObtenerInformacion para demostrar herencia y polimorfismo.

```
namespace PrestamosBiblioteca.Dominio;

public sealed class Lector : Persona
{
    public Lector(string codigo, string nombre, string correo, string telefono)
        : base(codigo, nombre, correo)
    {
        Telefono = string.IsNullOrWhiteSpace(telefono)
            ? throw new ArgumentException("El telefono es obligatorio.")
            : telefono.Trim();
    }

    public string Telefono { get; }
    public override string ObtenerTipo() => "Lector";

    public override string ObtenerInformacion() => $"{base.ObtenerInformacion()} | Tel: {Telefono}";
}
```

ARCHIVO 5 DE 11

Dominio/Libro.cs

Dominio (POO)

Entidad del catalogo: codigo, titulo, autor y cantidad de ejemplares. Encapsula y valida sus datos.

```
namespace PrestamosBiblioteca.Dominio;

public sealed class Libro
{
    public Libro(string codigo, string titulo, string autor, int ejemplares)
    {
        Codigo = string.IsNullOrWhiteSpace(codigo) ? throw new ArgumentException("El codigo del libro es obligatorio.") : codigo.Trim();
        Titulo = string.IsNullOrWhiteSpace(titulo) ? throw new ArgumentException("El titulo es obligatorio.") : titulo.Trim();
        Autor = string.IsNullOrWhiteSpace(autor) ? throw new ArgumentException("El autor es obligatorio.") : autor.Trim();
        if (ejemplares < 0) throw new ArgumentException("La cantidad de ejemplares no puede ser negativa.");
        Ejemplares = ejemplares;
    }

    public string Codigo { get; }
    public string Titulo { get; }
    public string Autor { get; }
    public int Ejemplares { get; }
}
```

ARCHIVO 6 DE 11

Dominio/Prestamo.cs

Dominio (POO)

Representa la relacion lector-libro con fechas. Valida que las fechas sean coherentes y expone el estado del prestamo.

```
namespace PrestamosBiblioteca.Dominio;

public sealed class Prestamo
{
    public Prestamo(int lectorId, int libroId, DateTime fechaPrestamo, DateTime fechaLimite, DateTime? fechaDevolucion = null)
    {
        if (lectorId <= 0) throw new ArgumentException("Debe seleccionar un lector.");
        if (libroId <= 0) throw new ArgumentException("Debe seleccionar un libro.");
        if (fechaLimite.Date < fechaPrestamo.Date) throw new ArgumentException("La fecha limite no puede ser anterior al prestamo.");
        if (fechaDevolucion.HasValue && fechaDevolucion.Value.Date < fechaPrestamo.Date)
            throw new ArgumentException("La devolucion no puede ser anterior al prestamo.");

        LectorId = lectorId;
        LibroId = libroId;
        FechaPrestamo = fechaPrestamo;
        FechaLimite = fechaLimite;
        FechaDevolucion = fechaDevolucion;
    }

    public int Id { get; set; }
    public int LectorId { get; }
    public int LibroId { get; }
    public DateTime FechaPrestamo { get; }
    public DateTime FechaLimite { get; }
    public DateTime? FechaDevolucion { get; }
    public bool EstaActivo => !FechaDevolucion.HasValue;
    public bool EstaVencido(DateTime hoy) => EstaActivo && FechaLimite.Date < hoy.Date;
}
```

ARCHIVO 7 DE 11

Persistencia/InicializadorBaseDatos.cs

Persistencia (SQLite)

Determina la ubicacion del archivo .db, abre la conexion con claves foraneas activas y ejecuta el script SQL de creacion de tablas.

```
using System.Reflection;
using Microsoft.Data.Sqlite;

namespace PrestamosBiblioteca.Persistencia;

internal sealed class InicializadorBaseDatos
{
    public string RutaBaseDatos { get; } = Environment.GetEnvironmentVariable("PRESTAMOS_BIBLIOTECA_DB")
        ?? Path.Combine(
            Environment.GetFolderPath(Environment.SpecialFolder.LocalApplicationData),
            "UMG",
            "PrestamosBiblioteca",
            "biblioteca.db");

    public string CadenaConexion => new SqliteConnectionStringBuilder
    {
        DataSource = RutaBaseDatos,
        ForeignKeys = true
    }.ToString();

    public void Inicializar()
    {
        var directorio = Path.GetDirectoryName(RutaBaseDatos);
        if (!string.IsNullOrEmpty(directorio)) Directory.CreateDirectory(directorio);
        using var conexion = new SqliteConnection(CadenaConexion);
        conexion.Open();
        using var comando = conexion.CreateCommand();
        comando.CommandText = LeerScriptTablas();
        comando.ExecuteNonQuery();
    }

    private static string LeerScriptTablas()
    {
        var ensamblado = Assembly.GetExecutingAssembly();
        var recurso = ensamblado.GetManifestResourceNames()
            .Single(nombre => nombre.EndsWith("crear_tablas.sql", StringComparison.OrdinalIgnoreCase));
        using var stream = ensamblado.GetManifestResourceStream(recurso)
            ?? throw new InvalidOperationException("No se encontro el script SQL de las tablas.");
        using var lector = new StreamReader(stream);
        return lector.ReadToEnd();
    }
}
```

ARCHIVO 8 DE 11

Persistencia/RepositorioBiblioteca.cs

Persistencia (SQLite)

Contiene las operaciones CRUD de lectores, libros y prestamos, la validacion de disponibilidad, el registro de devoluciones y el uso de transacciones. Todas las consultas usan parametros.

```
using System.Data;
using Microsoft.Data.Sqlite;
using PrestamosBiblioteca.Dominio;

namespace PrestamosBiblioteca.Persistencia;

internal sealed class RepositorioBiblioteca
{
    private readonly string _cadenaConexion;

    public RepositorioBiblioteca(string cadenaConexion) => _cadenaConexion = cadenaConexion;

    // CRUD de lectores
    public DataTable ObtenerLectores() => Consultar(
        "SELECT Id, Codigo, Nombre, Correo, Telefono FROM Lectores ORDER BY Nombre;");

    public void CrearLector(Lector lector) => Ejecutar(
        "INSERT INTO Lectores (Codigo, Nombre, Correo, Telefono) VALUES ($codigo, $nombre, $correo, $telefono);",
        ("$codigo", lector.Codigo), ("$nombre", lector.Nombre),
        ("$correo", lector.Correo), ("$telefono", lector.Telefono));

    public void ActualizarLector(int id, Lector lector) => Ejecutar(
        "UPDATE Lectores SET Codigo = $codigo, Nombre = $nombre, Correo = $correo, Telefono = $telefono WHERE Id = $id;",
        ("$codigo", lector.Codigo), ("$nombre", lector.Nombre), ("$correo", lector.Correo),
        ("$telefono", lector.Telefono), ("$id", id));

    public void EliminarLector(int id) => Ejecutar("DELETE FROM Lectores WHERE Id = $id;", ("$id", id));

    // CRUD de libros
    public DataTable ObtenerLibros() => Consultar(@"SELECT l.Id, l.Codigo, l.Titulo, l.Autor, l.Ejemplares,
        l.Ejemplares - (SELECT COUNT(*) FROM Prestamos p WHERE p.LibroId = l.Id AND p.FechaDevolucion IS NULL) AS Disponibles
        FROM Libros l ORDER BY l.Titulo;");

    public void CrearLibro(Libro libro) => Ejecutar(
        "INSERT INTO Libros (Codigo, Titulo, Autor, Ejemplares) VALUES ($codigo, $titulo, $autor, $ejemplares);",
        ("$codigo", libro.Codigo), ("$titulo", libro.Titulo),
        ("$autor", libro.Autor), ("$ejemplares", libro.Ejemplares));

    public void ActualizarLibro(int id, Libro libro)
    {
        using var conexion = AbrirConexion();
        using var comando = conexion.CreateCommand();
        comando.CommandText = @"UPDATE Libros SET Codigo = $codigo, Titulo = $titulo, Autor = $autor, Ejemplares = $ejemplares
            WHERE Id = $id AND $ejemplares >= (SELECT COUNT(*) FROM Prestamos WHERE LibroId = $id AND FechaDevolucion IS NULL);";
        comando.Parameters.AddWithValue("$codigo", libro.Codigo);
        comando.Parameters.AddWithValue("$titulo", libro.Titulo);
        comando.Parameters.AddWithValue("$autor", libro.Autor);
```

```
        comando.Parameters.AddWithValue("$ejemplares", libro.Ejemplares);
        comando.Parameters.AddWithValue("$id", id);
        if (comando.ExecuteNonQuery() == 0)
            throw new InvalidOperationException("No existe el libro o los ejemplares no pueden ser menos que los prestamos activos.");
    }

    public void EliminarLibro(int id) => Ejecutar("DELETE FROM Libros WHERE Id = $id;", ("$id", id));

    // CRUD y flujo de prÃ©stamos
    public DataTable ObtenerPrestamos() => Consultar(@"SELECT p.Id, p.LectorId, p.LibroId,
        l.Nombre AS Lector, b.Titulo AS Libro, p.FechaPrestamo, p.FechaLimite, p.FechaDevolucion,
        CASE WHEN p.FechaDevolucion IS NOT NULL THEN 'Devuelto'
             WHEN date(p.FechaLimite) < date('now', 'localtime') THEN 'Vencido'
             ELSE 'Prestado' END AS Estado
        FROM Prestamos p
        INNER JOIN Lectores l ON l.Id = p.LectorId
        INNER JOIN Libros b ON b.Id = p.LibroId
        ORDER BY p.FechaPrestamo DESC, p.Id DESC;");

    public void CrearPrestamo(Prestamo prestamo)
    {
        using var conexion = AbrirConexion();
        using var transaccion = conexion.BeginTransaction();
        ValidarDisponibilidad(conexion, transaccion, prestamo.LibroId, null);
        using var comando = conexion.CreateCommand();
        comando.Transaction = transaccion;
        comando.CommandText = "INSERT INTO Prestamos (LectorId, LibroId, FechaPrestamo, FechaLimite, FechaDevolucion) VALUES ($lector, $libro, $inicio, $limite, $devolucion);";
        AgregarParametrosPrestamo(comando, prestamo);
        comando.ExecuteNonQuery();
        transaccion.Commit();
    }

    public void ActualizarPrestamo(int id, Prestamo prestamo)
    {
        using var conexion = AbrirConexion();
        using var transaccion = conexion.BeginTransaction();
        if (prestamo.EstaActivo) ValidarDisponibilidad(conexion, transaccion, prestamo.LibroId, id);
        using var comando = conexion.CreateCommand();
        comando.Transaction = transaccion;
        comando.CommandText = @"UPDATE Prestamos SET LectorId = $lector, LibroId = $libro,
            FechaPrestamo = $inicio, FechaLimite = $limite, FechaDevolucion = $devolucion WHERE Id = $id;";
        AgregarParametrosPrestamo(comando, prestamo);
        comando.Parameters.AddWithValue("$id", id);
        if (comando.ExecuteNonQuery() == 0) throw new InvalidOperationException("No existe un prestamo con ese Id.");
        transaccion.Commit();
    }

    public void RegistrarDevolucion(int id, DateTime fechaDevolucion)
    {
        using var conexion = AbrirConexion();
        using var comando = conexion.CreateCommand();
        comando.CommandText = @"UPDATE Prestamos SET FechaDevolucion = $fecha
            WHERE Id = $id AND FechaDevolucion IS NULL AND date($fecha) >= date(FechaPrestamo);";
        comando.Parameters.AddWithValue("$fecha", fechaDevolucion.ToString("yyyy-MM-dd"));
        comando.Parameters.AddWithValue("$id", id);
        if (comando.ExecuteNonQuery() == 0)
            throw new InvalidOperationException("El prestamo no existe, ya fue devuelto o la fecha no es valida.");
    }

```

```
    public void EliminarPrestamo(int id) => Ejecutar("DELETE FROM Prestamos WHERE Id = $id;", ("$id", id));

    private static void ValidarDisponibilidad(SqliteConnection conexion, SqliteTransaction transaccion, int libroId, int? prestamoActualId)
    {
        using var comando = conexion.CreateCommand();
        comando.Transaction = transaccion;
        comando.CommandText = @"SELECT b.Ejemplares - (
            SELECT COUNT(*) FROM Prestamos p WHERE p.LibroId = b.Id AND p.FechaDevolucion IS NULL
            AND ($prestamoId IS NULL OR p.Id <> $prestamoId))
            FROM Libros b WHERE b.Id = $libroId;";
        comando.Parameters.AddWithValue("$libroId", libroId);
        comando.Parameters.AddWithValue("$prestamoId", prestamoActualId.HasValue ? prestamoActualId.Value : DBNull.Value);
        var disponibles = comando.ExecuteScalar();
        if (disponibles is null || disponibles is DBNull)
            throw new InvalidOperationException("El libro seleccionado no existe.");
        if (Convert.ToInt32(disponibles) <= 0)
            throw new InvalidOperationException("No hay ejemplares disponibles de ese libro.");
    }

    private static void AgregarParametrosPrestamo(SqliteCommand comando, Prestamo prestamo)
    {
        comando.Parameters.AddWithValue("$lector", prestamo.LectorId);
        comando.Parameters.AddWithValue("$libro", prestamo.LibroId);
        comando.Parameters.AddWithValue("$inicio", prestamo.FechaPrestamo.ToString("yyyy-MM-dd"));
        comando.Parameters.AddWithValue("$limite", prestamo.FechaLimite.ToString("yyyy-MM-dd"));
        comando.Parameters.AddWithValue("$devolucion", prestamo.FechaDevolucion.HasValue
            ? prestamo.FechaDevolucion.Value.ToString("yyyy-MM-dd")
            : DBNull.Value);
    }

    private DataTable Consultar(string sql)
    {
        using var conexion = AbrirConexion();
        using var comando = conexion.CreateCommand();
        comando.CommandText = sql;
        using var lector = comando.ExecuteReader();
        var tabla = new DataTable();
        tabla.Load(lector);
        return tabla;
    }

    private void Ejecutar(string sql, params (string Nombre, object Valor)[] parametros)
    {
        using var conexion = AbrirConexion();
        using var comando = conexion.CreateCommand();
        comando.CommandText = sql;
        foreach (var (nombre, valor) in parametros) comando.Parameters.AddWithValue(nombre, valor ?? DBNull.Value);
        comando.ExecuteNonQuery();
    }

    private SqliteConnection AbrirConexion()
    {
        var conexion = new SqliteConnection(_cadenaConexion);
        conexion.Open();
        return conexion;
    }
}
```

ARCHIVO 9 DE 11

Aplicacion/MenuBiblioteca.cs

Aplicacion (consola)

Menu interactivo: captura datos, invoca al repositorio, presenta tablas y controla errores con try-catch.

```
using System.Data;
using PrestamosBiblioteca.Dominio;
using PrestamosBiblioteca.Persistencia;

namespace PrestamosBiblioteca.Aplicacion;

internal sealed class MenuBiblioteca
{
    private readonly RepositorioBiblioteca _repositorio;
    private bool _continuar = true;

    public MenuBiblioteca(RepositorioBiblioteca repositorio) => _repositorio = repositorio;

    public void Ejecutar()
    {
        while (_continuar)
        {
            MostrarMenu();
            try
            {
                var entrada = Console.ReadLine();
                if (entrada is null) break;
                switch (entrada.Trim())
                {
                    case "1": CrearLector(); break;
                    case "2": ListarLectores(); break;
                    case "3": ActualizarLector(); break;
                    case "4": EliminarLector(); break;
                    case "5": CrearLibro(); break;
                    case "6": ListarLibros(); break;
                    case "7": ActualizarLibro(); break;
                    case "8": EliminarLibro(); break;
                    case "9": CrearPrestamo(); break;
                    case "10": ListarPrestamos(); break;
                    case "11": ActualizarPrestamo(); break;
                    case "12": RegistrarDevolucion(); break;
                    case "13": EliminarPrestamo(); break;
                    case "14": DemostrarPolimorfismo(); break;
                    case "0": _continuar = false; break;
                    default: Console.WriteLine("Seleccione una opcion valida."); break;
                }
            }
            catch (Exception error)
            {
                Console.ForegroundColor = ConsoleColor.Red;
                Console.WriteLine($"Error controlado con try-catch: {error.Message}");
                Console.ResetColor();
            }

            if (_continuar) Pausar();
        }
    }

    private static void MostrarMenu()
```

```
    {
        if (!Console.IsOutputRedirected) Console.Clear();
        Console.ForegroundColor = ConsoleColor.Cyan;
        Console.WriteLine("=============================================");
        Console.WriteLine("       SISTEMA DE PRESTAMOS DE BIBLIOTECA");
        Console.WriteLine("=============================================");
        Console.ResetColor();
        Console.WriteLine("LECTORES: 1 Crear | 2 Listar | 3 Actualizar | 4 Eliminar");
        Console.WriteLine("LIBROS:   5 Crear | 6 Listar | 7 Actualizar | 8 Eliminar");
        Console.WriteLine("PRESTAMOS: 9 Crear | 10 Listar | 11 Actualizar | 12 Devolver | 13 Eliminar");
        Console.WriteLine("POO: 14 Demostrar polimorfismo | 0 Salir");
        Console.Write("Opcion: ");
    }

    private void CrearLector()
    {
        _repositorio.CrearLector(LeerLector());
        Console.WriteLine("Lector creado correctamente.");
    }

    private void ListarLectores() => MostrarTabla(_repositorio.ObtenerLectores(), "LECTORES");

    private void ActualizarLector()
    {
        _repositorio.ActualizarLector(LeerId("Id del lector a actualizar"), LeerLector());
        Console.WriteLine("Lector actualizado correctamente (si el Id existe).");
    }

    private void EliminarLector()
    {
        _repositorio.EliminarLector(LeerId("Id del lector a eliminar"));
        Console.WriteLine("Lector eliminado; sus prestamos tambien se eliminan.");
    }

    private void CrearLibro()
    {
        _repositorio.CrearLibro(LeerLibro());
        Console.WriteLine("Libro creado correctamente.");
    }

    private void ListarLibros() => MostrarTabla(_repositorio.ObtenerLibros(), "LIBROS Y EJEMPLARES
DISPONIBLES");

    private void ActualizarLibro()
    {
        _repositorio.ActualizarLibro(LeerId("Id del libro a actualizar"), LeerLibro());
        Console.WriteLine("Libro actualizado correctamente.");
    }

    private void EliminarLibro()
    {
        _repositorio.EliminarLibro(LeerId("Id del libro a eliminar"));
        Console.WriteLine("Libro eliminado; sus prestamos tambien se eliminan.");
    }

    private void CrearPrestamo()
    {
        MostrarTabla(_repositorio.ObtenerLectores(), "LECTORES");
        MostrarTabla(_repositorio.ObtenerLibros(), "LIBROS");
        var inicio = DateTime.Today;
        var limite = LeerFecha("Fecha limite (AAAA-MM-DD)");
        var prestamo = new Prestamo(LeerId("Id del lector"), LeerId("Id del libro"), inicio, limite);
        _repositorio.CrearPrestamo(prestamo);
```

```
        Console.WriteLine("Prestamo registrado; el ejemplar queda ocupado hasta su devolucion.");
    }

    private void ListarPrestamos() => MostrarTabla(_repositorio.ObtenerPrestamos(), "PRESTAMOS Y ESTADO");

    private void ActualizarPrestamo()
    {
        MostrarTabla(_repositorio.ObtenerLectores(), "LECTORES");
        MostrarTabla(_repositorio.ObtenerLibros(), "LIBROS");
        var id = LeerId("Id del prestamo a actualizar");
        var lectorId = LeerId("Id del lector");
        var libroId = LeerId("Id del libro");
        var inicio = LeerFecha("Fecha de prestamo (AAAA-MM-DD)");
        var limite = LeerFecha("Fecha limite (AAAA-MM-DD)");
        _repositorio.ActualizarPrestamo(id, new Prestamo(lectorId, libroId, inicio, limite));
        Console.WriteLine("Prestamo actualizado correctamente.");
    }

    private void RegistrarDevolucion()
    {
        _repositorio.RegistrarDevolucion(LeerId("Id del prestamo a devolver"), DateTime.Today);
        Console.WriteLine("Devolucion registrada; el ejemplar vuelve a estar disponible.");
    }

    private void EliminarPrestamo()
    {
        _repositorio.EliminarPrestamo(LeerId("Id del prestamo a eliminar"));
        Console.WriteLine("Prestamo eliminado.");
    }

    private static void DemostrarPolimorfismo()
    {
        Persona persona = new Lector("L-DEMO", "Ana Lopez", "ana@umg.edu.gt", "5555-0101");
        Console.WriteLine("Referencia Persona que contiene una instancia de Lector:");
        Console.WriteLine(persona.ObtenerInformacion());
    }

    private static Lector LeerLector() => new(
        LeerTexto("Codigo"), LeerTexto("Nombre"), LeerTexto("Correo"), LeerTexto("Telefono"));

    private static Libro LeerLibro() => new(
        LeerTexto("Codigo"), LeerTexto("Titulo"), LeerTexto("Autor"), LeerEntero("Cantidad de
ejemplares"));

    private static int LeerId(string mensaje) => LeerEntero(mensaje);

    private static int LeerEntero(string mensaje)
    {
        Console.Write($"{mensaje}: ");
        if (!int.TryParse(Console.ReadLine(), out var valor)) throw new ArgumentException("Ingrese un
numero entero valido.");
        return valor;
    }

    private static DateTime LeerFecha(string mensaje)
    {
        Console.Write($"{mensaje}: ");
        if (!DateTime.TryParse(Console.ReadLine(), out var fecha)) throw new ArgumentException("Ingrese una
fecha valida.");
        return fecha.Date;
    }

    private static string LeerTexto(string mensaje)
```

```
    {
        Console.Write($"{mensaje}: ");
        var texto = Console.ReadLine();
        return string.IsNullOrWhiteSpace(texto)
            ? throw new ArgumentException($"El campo '{mensaje}' es obligatorio.")
            : texto.Trim();
    }

    private static void MostrarTabla(DataTable tabla, string titulo)
    {
        Console.WriteLine($"\n=== {titulo} ===");
        if (tabla.Rows.Count == 0)
        {
            Console.WriteLine("No hay registros.");
            return;
        }

        foreach (DataColumn columna in tabla.Columns) Console.Write($"{columna.ColumnName,-22}");
        Console.WriteLine();
        foreach (DataRow fila in tabla.Rows)
        {
            foreach (var valor in fila.ItemArray) Console.Write($"{valor,-22}");
            Console.WriteLine();
        }
    }

    private static void Pausar()
    {
        if (Console.IsInputRedirected) return;
        Console.WriteLine("\nPresione Enter para continuar...");
        Console.ReadLine();
    }
}
```

ARCHIVO 10 DE 11

BaseDatos/crear_tablas.sql

Base de datos

Script SQL con las tablas Lectores, Libros y Prestamos, claves foraneas, restricciones CHECK e indices.

```sql
PRAGMA foreign_keys = ON;

CREATE TABLE IF NOT EXISTS Lectores (
    Id INTEGER PRIMARY KEY AUTOINCREMENT,
    Codigo TEXT NOT NULL UNIQUE,
    Nombre TEXT NOT NULL,
    Correo TEXT NOT NULL UNIQUE,
    Telefono TEXT NOT NULL
);

CREATE TABLE IF NOT EXISTS Libros (
    Id INTEGER PRIMARY KEY AUTOINCREMENT,
    Codigo TEXT NOT NULL UNIQUE,
    Titulo TEXT NOT NULL,
    Autor TEXT NOT NULL,
    Ejemplares INTEGER NOT NULL CHECK (Ejemplares >= 0)
);

CREATE TABLE IF NOT EXISTS Prestamos (
    Id INTEGER PRIMARY KEY AUTOINCREMENT,
    LectorId INTEGER NOT NULL,
    LibroId INTEGER NOT NULL,
    FechaPrestamo TEXT NOT NULL,
    FechaLimite TEXT NOT NULL,
    FechaDevolucion TEXT NULL,
    CHECK (date(FechaLimite) >= date(FechaPrestamo)),
    CHECK (FechaDevolucion IS NULL OR date(FechaDevolucion) >= date(FechaPrestamo)),
    FOREIGN KEY (LectorId) REFERENCES Lectores(Id) ON DELETE CASCADE,
    FOREIGN KEY (LibroId) REFERENCES Libros(Id) ON DELETE CASCADE
);

CREATE INDEX IF NOT EXISTS IX_Prestamos_LectorId ON Prestamos(LectorId);
CREATE INDEX IF NOT EXISTS IX_Prestamos_LibroId_Estado ON Prestamos(LibroId, FechaDevolucion);
```

ARCHIVO 11 DE 11

README.md

Documentacion

Instrucciones de instalacion, ejecucion, descripcion de tablas, reglas de negocio y recorrido sugerido para clase.

```
# Sistema de Prestamos de Biblioteca

Ejemplo didactico de consola en C# para registrar lectores, libros y prestamos en SQLite. Recorre el flujo desde las tablas y sus relaciones hasta las operaciones CRUD, las reglas de disponibilidad y la devolucion de ejemplares.

## Requisitos y ejecucion

- .NET 6 SDK o posterior.
- Conexion a Internet al restaurar por primera vez el paquete NuGet.

Desde esta carpeta:

```powershell
dotnet restore
dotnet run
```

`Microsoft.Data.Sqlite` esta declarado en `PrestamosBiblioteca.csproj`; NuGet descarga automaticamente el proveedor y las bibliotecas necesarias. No se deben copiar DLLs manualmente. Al iniciar, el programa crea la base de datos y sus tablas si no existen.

La base se crea en `%LOCALAPPDATA%\UMG\PrestamosBiblioteca\biblioteca.db`. Para pruebas, puede establecerse la variable de entorno `PRESTAMOS_BIBLIOTECA_DB` con otra ruta completa. Para reiniciar el ejemplo, cierre el programa y elimine el archivo `.db` correspondiente.

Tablas y relaciones

- `Lectores`: personas que pueden solicitar libros.
- `Libros`: catalogo con cantidad total de ejemplares.
- `Prestamos`: relacion de un lector con un libro, fechas del prestamo y eventual devolucion.
- Las claves foraneas relacionan prestamos con lectores y libros.
- Los ejemplares disponibles se calculan restando los prestamos activos del total.

Revise el esquema completo en `BaseDatos/crear_tablas.sql`.

CRUD y reglas

El menu permite crear, listar, actualizar y eliminar lectores, libros y prestamos, ademas de registrar devoluciones. No se permite prestar cuando no hay ejemplares disponibles. Un ejemplar vuelve a estar disponible al registrar su devolucion. El listado informa si cada prestamo esta `Prestado`, `Vencido` o `Devuelto`.

Las consultas estan separadas en `Persistencia/RepositorioBiblioteca.cs` y usan parametros SQL. El menu interactivo esta en `Aplicacion/MenuBiblioteca.cs`.

POO y manejo de errores

- `Persona` es abstracta y `Lector` hereda de ella.
- `Persona persona = new Lector(...)` y la opcion 14 muestran polimorfismo.
- `Libro` y `Prestamo` encapsulan sus datos y validan reglas de negocio.
- La capa de persistencia separa las entidades de la base de datos.
- `try-catch` en el menu controla datos incorrectos, restricciones SQLite y reglas del sistema sin cerrar la aplicacion.
```

```
## Recorrido sugerido para clase

1. Leer `BaseDatos/crear_tablas.sql` y ubicar las claves foraneas.
2. Crear un lector y un libro con un ejemplar.
3. Crear un prestamo y consultar el listado: los ejemplares disponibles deben pasar a cero.
4. Intentar otro prestamo del mismo libro y observar el error controlado.
5. Registrar la devolucion y consultar nuevamente la disponibilidad.
6. Probar actualizar/eliminar datos y la opcion 14 para observar polimorfismo.
```

9. Conclusiones

El Sistema de Prestamos de Biblioteca cubre el circuito completo de una base de datos aplicada a un caso de uso real: desde el diseno del esquema relacional hasta la ejecucion de operaciones CRUD y reglas de negocio con manejo de errores. La separacion por capas, el uso de consultas parametrizadas y las transacciones constituyen buenas practicas que el estudiante puede reutilizar en proyectos posteriores.